

Proyecto Final – Smart Parking Albacete

Internet de las Cosas y sus Aplicaciones (cód. 311482)

alonso.marcos@alu.uclm.es

Máster en Big Data y Computación en la Nube – UCLM

Mayo 2026

0. Resumen ejecutivo

El Ayuntamiento de Albacete licita una solución de **smart parking** que monitorice en tiempo real las plazas del entorno universitario (BBOX 38.976059, -1.858728 → 38.983215, -1.846111, $\approx 0,55 \text{ km}^2$) y exponga la disponibilidad a apps de movilidad, vehículos autónomos (vía C-V2X) y paneles informativos. TECO S.L. (rol del autor) responde con una arquitectura end-to-end en cuatro capas: (1) sensórica magnética AMR por plaza más cámaras ANPR puntuales en accesos; (2) conectividad NB-IoT como red principal y LoRaWAN/5G como complementos; (3) plataforma cloud AWS serverless (IoT Core + Lambda + DynamoDB + API Gateway); (4) aplicaciones API REST OpenAPI 3.0 (salida GeoJSON) y dashboard Streamlit.

El prototipo cubre el flujo extremo a extremo con 40 plazas simuladas en 4 sub-zonas reales (Z1-CAMPUS, Z2-DEPORTIVO, Z3-SANITARIO, Z4-RESIDENCIAL), desplegado en AWS Academy Learner Lab con scripts boto3 idempotentes. La latencia medida sensor→API es $< 2 \text{ s}$, el coste cloud del piloto $< 5 \text{ USD/mes}$ y la arquitectura escala linealmente a 10 000 plazas con coste cloud aproximado de **6 céntimos/plaza/mes**.

1. Descripción del problema y entorno

1.1 Contexto

Entre el 15 % y el 30 % del tráfico urbano (Shoup, 2005) lo generan vehículos buscando aparcamiento, con impacto directo en tiempo, emisiones y confort. Albacete ($\approx 175\,000$ hab.) propone como piloto el corredor entre Campus UCLM, Estadio Carlos Belmonte, Hospital Universitario y la zona residencial sur de la AB-20, alineado con el Pacto Verde Europeo y la llegada de la movilidad autónoma.

1.2 Actores y zona piloto

Actor	Rol
Ayuntamiento de Albacete	Cliente final / licitador
TECO S.L.	Adjudicatario (autor)
Concejalía de Movilidad	Explota KPIs municipales
Vehículos autónomos / OEM	Consumidores futuros vía API + C-V2X
Operador ANPR	Provee vídeo en accesos

Sub-zonas operativas: **Z1-CAMPUS** (UCLM, picos 7:30-10:00 y 16-18 h), **Z2-DEPORTIVO** (Estadio Belmonte $\approx 17\,500$ espectadores, correlación con calendario), **Z3-SANITARIO** (Hospital + Facultades, demanda 24/7) y **Z4-RESIDENCIAL** (Av. Mancha y Olimpia, patrón nocturno).

1.3 Supuestos realistas

Supuesto	Valor	Justificación
Plazas piloto	≈ 500	BBOX 800×800 m, aparcamiento en cordón
Plazas/sub-zona	~ 125	Equilibrio simplicidad / representatividad

Supuesto	Valor	Justificación
Rotación diaria	4-8 (Z3), 3-5 (Z1), 1-3 (Z4), picos (Z2)	Datos comparables españoles
Cobertura NB-IoT	100 % en la BBOX	Operadores nacionales con LTE completo
Disponibilidad piloto	99,5 %	Objetivo productivo 99,9 %
Latencia E2E objetivo	≤ 5 s	Compatible con guía humana y C-V2X
Cadencia	1 evento/cambio + HB 5 min	Minimiza OPEX LPWAN
Vida útil sensor	5-7 años	Habitual AMR comerciales

1.4 Restricciones y riesgos identificados

Imposibilidad legal de cubrir cada plaza con cámaras (RGPD); instalación bajo asfalto con corte nocturno; rango térmico -5/+40 °C (sensores IP68); cohabitación con otros servicios IoT municipales; una sola región AWS (`us-east-1`) por las 3 h del AWS Academy Learner Lab.

Riesgo	Prob.	Imp.	Mitigación
Falsos positivos AMR por motos cercanas	Media	Bajo	Debounce edge + umbral de confidence
Pérdida cobertura NB-IoT puntual	Baja	Medio	Buffer local + reintento exponencial
Sensores robados / vandalizados	Baja	Bajo	Caja blindada, alarma, coste unitario bajo
Saturación API en eventos deportivos	Media	Medio	Throttling API GW + caché ante terceros
Fuga de datos personales	Muy baja	Alto	Política sin matrícula ni PII
Caducidad credenciales lab (3 h)	Alta (acad.)	Alto	Scripts idempotentes, teardown rápido

2. Requisitos

2.1 Funcionales

ID	Requisito	Criterio de aceptación
RF-01	Detectar libre/ocupada por plaza	Reflejo en almacén operacional ≤ 5 s
RF-02	Asociar evento a plaza y sub-zona	Payload con <code>spotId</code> y <code>zoneId</code>
RF-03	Transmitir desde sensor/simulador	MQTT/TLS contra AWS IoT Core
RF-04	Mantener estado actual	Tabla DynamoDB <code>parking-state</code>
RF-05	Dashboard de disponibilidad	Streamlit, refresco ≤ 10 s
RF-06	API REST para terceros	API Gateway + OpenAPI 3.0
RF-07	Análisis por sub-zona	Parámetro <code>?zone=</code> , endpoint <code>/zones</code>
RF-08	Salida cartográfica	<code>?format=geojson</code> (RFC 7946)
RF-09	Agregados libres/ocupadas/%	Lambda agregador actualiza <code>zone-kpis</code>
RF-10	Histórico operacional	Serie temporal en <code>zone-kpis</code>
RF-11	Detectar sensores caídos	Auditoría de <code>lastUpdated</code> (alerta: futuro)

2.2 No funcionales

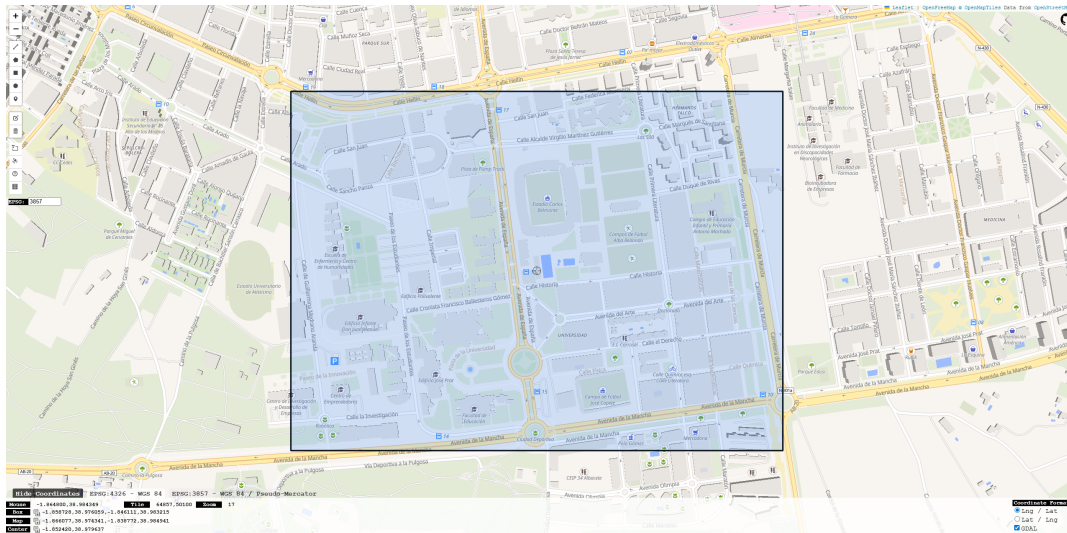


Figura 1: Bounding box de la zona piloto sobre el entorno universitario de Albacete (BBOX 38.976059, -1.858728 → 38.983215, -1.846111, $\approx 0,55 \text{ km}^2$). Fuente: bboxfinder.com.

ID	Requisito	Criterio
RNF-01	Latencia E2E $\leq 5 \text{ s}$	Medido $\leq 2 \text{ s}$
RNF-02	API p95 $\leq 1 \text{ s}$	Verificado
RNF-03	≥ 10 eventos/s sin throttling	DynamoDB on-demand
RNF-04	Escala 500 → 10 000 sin rediseño	Plan en cap. 8
RNF-05	Servicios serverless/gestionados	IoT Core, Lambda, DynamoDB, API GW
RNF-06	Disponibilidad $\geq 99,5 \%$	SLA IoT Core 99,9 %, Lambda 99,95 %
RNF-07	Tolerar pérdida temporal de red	Backoff MQTT + buffer local
RNF-08	Cifrado en tránsito	MQTT/TLS 1.2 mTLS, HTTPS
RNF-09	Identidad por dispositivo	Thing + certificado X.509
RNF-10	Throttling API	API Gateway + API keys
RNF-11	Sin PII	Modelo sin matrícula
RNF-12	Despliegue reproducible	Scripts boto3 idempotentes
RNF-13	Coste cloud piloto $< 10 \text{ USD/mes}$	Estimación cap. 9
RNF-14	API autodescriptiva	OpenAPI 3.0.1 + GeoJSON
RNF-15	Cumplimiento RGPD	Cap. 7

Restricciones impuestas: RST-01 AWS obligatorio; RST-02 credenciales lab 3 h; RST-03 solo LabRole; RST-04 defensa oral; RST-05 documentación en castellano; RST-06 Windows estándar del autor.

3. Capa de telemetría

3.1 Comparativa de tecnologías y productos de referencia

Tec.	Producto comercial	Precisión	Coste/ud	Consumo	Clima	Privacidad	
AMR mag.	Nedap Smart PS101, Bosch	SENSIT, Parking	>95 %	60-120 €	5-10 μ A	Excelente IP68	Muy alta

Tec.	Producto comercial	Precisión	Coste/ud	Consumo	Clima	Privacidad
Ultrasón.	Libelium Smart Parking, Worldsensing Fastprk-U	Alta int., baja ext.	80-150 €	0,5-1 mA	Media	Muy alta
Radar mmW	TI IWR6843, Vayyar	Muy alta	200-500 €	5-50 mA (red)	Excelente	Alta
LIDAR	Velodyne Puck, Quaternary M8	Muy alta	1500-5000 €	Decenas W	Buena	Alta
Cám. ANPR	YOLO/EfficientDet sobre IP cam	Muy alta + matr.	500-1200 €	Red	Media	Baja

3.2 Decisión razonada

Se adopta una **arquitectura híbrida**:

- **Sensor principal: AMR magnético bajo asfalto, uno por plaza.** Sellado IP68, calibración in situ, debounce 2-3 s, vida 5-7 años, ≈ 75 €/ud. Gana en consumo y coste a ultrasónico, radar y LIDAR, y en privacidad y simplicidad a la cámara.
- **Complementario: 6-8 cámaras ANPR puntuales** en accesos (N-430, C. San Juan, C. Imperial, Av. de la Mancha) para conteo agregado y futuro control de zonas de bajas emisiones; nunca para detección plaza a plaza (mitiga el riesgo RGPD).
- **Sensores ambientales** (NO₂, PM2.5, CO₂, ruido) 1 cada ~ 150 plazas como analítica complementaria.

3.3 Modelo del nodo y payload

Cada plaza es un Thing en AWS IoT Core con atributos `spotId`, `zoneId`, `street`, `lat`, `lon`. Payload MQTT (topic `parking/{zoneId}/spot/{spotId}/status`):

```
{
  "spotId": "ALB-Z1-001", "zoneId": "Z1-CAMPUS", "street": "C. Imperial",
  "lat": 38.97765, "lon": -1.85745, "status": "occupied",
  "batteryLevel": 92.2, "confidence": 0.94,
  "sensorType": "magnetic", "timestamp": 1778929200072
}
```

Política de envío: cambio de estado inmediato tras debounce, heartbeat cada 5 min, auto-test diario, alarma de manipulación y aviso de batería baja (< 20 %).

3.4 Calibración, instalación y mantenimiento

Calibración inicial en taller (linealización del magnetómetro) y final *in situ* (medida del campo terrestre sin vehículo), 5-10 min/ud. Instalación nocturna por cuadrillas de 2 operarios con maquinaria menor, rendimiento típico 30-40 plazas/noche. Mantenimiento: revisión por muestreo cada 6 meses; sustitución de pila prevista a los 5 años; firmware actualizable OTA vía LPWAN cuando la pila lo permite o por visita técnica programada.

4. Conectividad

4.1 Comparativa

Tec.	Latencia	Cobertura	CAPEX	OPEX	Consumo	Idoneidad
NB-IoT	1,6-10 s	Op. nacional	0 €	0,3-1 €/SIM/mes	Muy bajo	Sensores principales

Tec.	Latencia	Cobertura	CAPEX	OPEX	Consumo	Idoneidad
LoRaWAN	1-2 s	Red privada	600-1500 €/gw	~0	Muy bajo	Plan B
Sigfox	—	EU	—	—	Muy bajo	Descartada
Wi-Fi/Eth.	<100 ms	Local	—	—	Alto	Gateways/cámaras
5G NR	<10 ms URLLC	Parcial	—	Medio	Alto	Backhaul, V2X
C-V2X	—	LTE/5G	—	—	—	Salida a vehículos

4.2 Decisión

Capa	Tecnología	Razón
Sensor → cloud	NB-IoT	Cobertura municipal sin infraestructura propia; OPEX bajo; vida multianual
Plan B	LoRaWAN	Donde el ayuntamiento ya tenga gateways; cero coste por mensaje
Backhaul gw/cámara	Ethernet/fibra/5G	Volumen y alimentación eléctrica
Vehículo autónomo	API REST + C-V2X	Estándar, desacoplado

Esta combinación reproduce la práctica habitual en proyectos municipales españoles ya operativos: Santander (SmartSantander con sensores AMR sobre red propia y troncal IP), Málaga (smart parking con LoRaWAN privado en zona azul) y Pontevedra (NB-IoT en aparcamientos disuasorios). En el ámbito europeo, Niza y Lyon mantienen despliegues mixtos NB-IoT + LoRaWAN con APIs abiertas similares a la propuesta.

4.3 Plan de despliegue

Fase 0 (semanas 1-4): mapa exacto de las ≈ 500 plazas con coordenadas y prioridades, coordinación con Movilidad y EMT para cortes nocturnos, acuerdo con operador NB-IoT (tarifa SIM M2M, APN privado) y acuerdo con el responsable de la red WAN municipal. **Fase 1 (semanas 5-10):** 3 gateways edge sobre farolas o mobiliario urbano alimentados por alumbrado público, 20 sensores piloto en Z1-CAMPUS, despliegue del backend AWS y 2-3 semanas de validación operativa. **Fase 2 (semanas 11-20):** instalación nocturna de los ≈ 500 sensores restantes (~ 35 /noche), 6-8 cámaras ANPR en accesos, 4 nodos ambientales, onboarding masivo con IoT Provisioning Templates. **Fase 3 (meses 6-12):** KPIs de servicio (uptime, latencia E2E, tasa de falsos positivos), iteración del agregador, integración con paneles DMS municipales. **Fase 4 (año 2):** escalado por barrios e integración C-V2X (RSU/MEC).

4.4 Estimación de tráfico (piloto)

$500 \text{ plazas} \times 5 \text{ cambios/día} = 2500 \text{ eventos}$; $500 \times 288 \text{ heartbeats/día} = 144000$; total ≈ 150000 mensajes/día ($\sim 85 \text{ MB}$). Pico evento deportivo: hasta 1000 cambios/min durante 15 min. Asumible por NB-IoT y por la capa cloud (cap. 8).

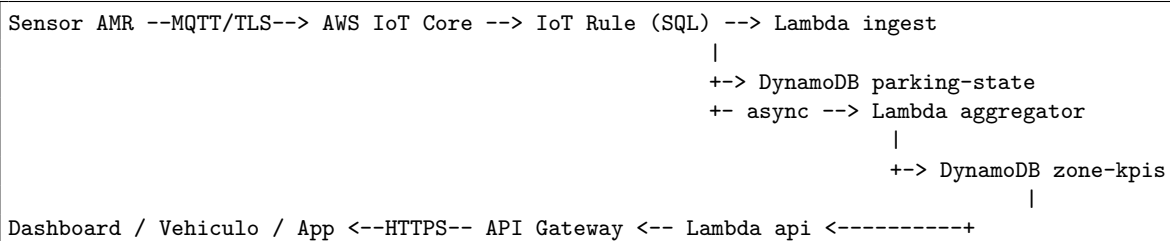
5. Arquitectura cloud en AWS

5.1 Servicios usados

Servicio	Rol
AWS IoT Core	Broker MQTT/TLS, registry, Things, certificados, IoT Rules
AWS IoT Rule	Filtrado SQL del topic y enrutado a Lambda
AWS Lambda	Ingesta, agregación y API (serverless)
Amazon DynamoDB	Estado actual + serie temporal (on-demand)
Amazon API Gateway	Fachada REST + throttling + cuotas
Amazon CloudWatch	Logs y métricas
IAM (LabRole)	Identidad de las Lambdas (restricción Academy)

Diseñados pero no desplegados (lab o coste/beneficio): IoT Greengrass, S3 raw, Cognito.

5.2 Diagrama de arquitectura



5.3 Flujo extremo a extremo

1. Sensor publica MQTT/TLS sobre `parking/{zoneId}/spot/{spotId}/status`.
2. IoT Core autentica vía mTLS X.509 y entrega al broker.
3. La IoT Rule (`SELECT * FROM 'parking/+/spot/+/status'`) dispara la Lambda ingest.
4. La Lambda ingest hace UPSERT en `parking-state` y, si cambia el estado, invoca de forma asíncrona la Lambda agregador con el `zoneId`.
5. La Lambda agregador hace Scan filtrado por zona, calcula libres/ocupadas/ratio y escribe una fila en `zone-kpis` con `windowEnd`.
6. API Gateway expone `/spots`, `/spots/{id}`, `/zones`, `/zones/{id}/kpis` y los integra con la Lambda api (JSON o GeoJSON).
7. Cliente (dashboard o tercero) consume por HTTPS; CloudWatch recoge logs.

5.4 Modelado de Things y policy

Todos los sensores pertenecen al Thing Type `smart-parking-albacete-sensor-type` y al Thing Group `smart-parking-albacete-fleet` (facilita acciones masivas). En el piloto comparten un único certificado y la policy `smart-parking-albacete-sensor-policy`, que autoriza solo `Connect/Publish/Subscribe` sobre topics `parking/*`. En producción se generaría un certificado por dispositivo (Fleet Provisioning Templates) para revocaciones granulares.

5.5 Reparto edge / cloud

Edge (diseño con IoT Greengrass v2): filtrado de ruido del magnetómetro, debounce y detección de cambios reales, caché de hasta 24 h ante caída de conectividad, agregación simple por zona, procesamiento ANPR local con envío de solo metadatos (hash, dirección, recuento). **Cloud (implementado):** normalización del payload, persistencia de estado y agregados, APIs internas y externas, seguridad, observabilidad.

5.6 Nota sobre interoperabilidad (FIWARE, opcional)

El enunciado exige AWS y contempla la posibilidad de proponer una integración híbrida con **FIWARE** si se justifica. Esta línea es relevante en el contexto europeo porque numerosos pliegos municipales (Open & Agile Smart Cities, proyectos H2020/Horizon Europe) demandan compatibilidad con el modelo NGSI-v2 / NGSI-LD y con los Smart Data Models públicos del catálogo

`smart-data-models.github.io` (`ParkingSpot`, `ParkingZone`, `ParkingSensor`). Para fases posteriores se prevé exponer un endpoint adicional alimentado por la misma Lambda de ingesta (con un POST al Context Broker Orion tras escribir en DynamoDB), de modo que un tercero pueda consumir por NGSI-v2 sin tocar el resto de la arquitectura. En el piloto **no se despliega** Orion porque añade contenedores y puntos de fallo que no aportan valor académico extra dado el límite de 3 h del lab, y porque el contrato REST + GeoJSON ya cubre los requisitos funcionales exigidos.

5.7 Implementación efectiva del prototipo

Recurso	Nombre	Estado
Thing Type / Group	<code>*-sensor-type</code> / <code>*-fleet</code>	Creados
Things	ALB-Z1-001 ... ALB-Z4-010 (40 ud.)	Creados
Certificado + Policy	Compartidos en piloto	Creados
IoT Topic Rule	<code>smart_parking_albacete_ingest_rule</code>	Activa
DynamoDB	<code>*-state</code> , <code>*-zone-kpis</code>	ACTIVE
Lambdas	<code>*-ingest</code> , <code>*-aggregator</code> , <code>*-api</code>	Desplegadas
API Gateway	<code>smart-parking-albacete-api</code> (id 85fbp0svzc)	Stage prod

6. Modelo de datos y APIs

6.1 Modelado físico en DynamoDB

Tabla `*-state` (estado actual, PK `spotId`): `zoneId`, `street`, `lat`, `lon`, `status` (free/occupied/unknown), `batteryLevel`, `confidence`, `sensorType`, `lastUpdated`. Modo `PAY_PER_REQUEST`. Operaciones principales: `PutItem` (UPSERT) desde la Lambda ingest y `Scan` filtrado por `zoneId` desde la Lambda agregador y la Lambda api. En escenario ciudad: GSI `zoneId-status-index` para sustituir el `Scan` por `Query`.

Tabla `*-zone-kpis` (serie temporal, PK `zoneId`, SK `windowEnd ISO 8601`): `totalSpots`, `freeSpots`, `occupiedSpots`, `unknownSpots`, `occupancyRate`, `computedAtMs`. Permite consultas naturales del tipo *últimos N KPIs de Z1*.

6.2 Contrato OpenAPI 3.0

Mét.	Ruta	Descripción	Parámetros
GET	<code>/spots</code>	Lista plazas con estado actual	<code>?zone=</code> , <code>?format=geojson</code>
GET	<code>/spots/{spotId}</code>	Detalle de una plaza	—
GET	<code>/zones</code>	Sub-zonas con KPIs vivos	—
GET	<code>/zones/{zoneId}/kpis</code>	Serie temporal de KPIs	<code>?limit=</code> (def. 100)

Todos los endpoints devuelven `application/json; charset=utf-8`. CORS habilitado para el dashboard local. Ejemplo de respuesta de `GET /zones`:

```
{
  "count": 4,
  "items": [
    {
      "zoneId": "Z1-CAMPUS",
      "total": 10,
      "free": 6,
      "occupied": 4,
      "occupancyRate": 0.4
    },
    {
      "zoneId": "Z2-DEPORTIVO",
      "total": 10,
      "free": 4,
      "occupied": 6,
      "occupancyRate": 0.6
    },
    {
      "zoneId": "Z3-SANITARIO",
      "total": 10,
      "free": 6,
      "occupied": 4,
      "occupancyRate": 0.4
    },
    {
      "zoneId": "Z4-RESIDENCIAL",
      "total": 9,
      "free": 7,
      "occupied": 2,
      "occupancyRate": 0.222
    }
  ]
}
```

GeoJSON RFC 7946 disponible vía `?format=geojson` para consumo directo desde mapas y OEM (FeatureCollection con Point geometry y properties por plaza).

6.3 APIs internas vs externas e idempotencia

La API pública es de solo lectura, con rate-limit y opcional API key/Cognito. La API interna (operador) permite forzar **unknown** para plazas en obras y queda protegida con IAM y red privada (documentada, no desplegada). La Lambda ingest hace **PutItem** idempotente; en escenarios con orden estricto se introduce **ConditionExpression: lastUpdated <:ts**. El versionado se hará en el path (/v1/, /v2/) al pasar a productivo.

6.4 Catálogo de KPIs

KPI	Periodicidad	Origen
% ocupación por sub-zona	Tiempo real	Lambda agregador
% ocupación por calle	A demanda	Scan sobre *-state
Plazas libres absolutas por sub-zona	Tiempo real	Lambda agregador
Tiempo medio de ocupación por plaza	Diario	Trabajo futuro
Rotación por plaza	Diario	Trabajo futuro
Sensores caídos / batería baja	Cada hora	Trabajo futuro (auditoría)
Saturación pico (máx. %)	Diario	Trabajo futuro

7. Seguridad y privacidad

7.1 Modelo de amenazas (resumen)

Activo	Amenaza	Mitigación
Sensor físico	Manipulación, robo	Caja blindada, alarma de movimiento
Identidad sensor	Suplantación	Certif. por dispositivo + rotación + revocación
Canal sensor↔cloud	MITM, eavesdropping	mTLS TLS 1.2
Cloud (datos)	Acceso no autorizado	IAM mínimo privilegio + auditoría
API pública	Abuso, DoS	Throttling, cuotas, API keys, WAF
Datos personales	Inferencia	Sin matrículas ni datos del conductor
Firmware OTA	Backdoor	Firma criptográfica y validación en dispositivo

7.2 Implementación

Dispositivo: un Thing por plaza con certificado X.509 compartido en el piloto y policy restrictiva (**Connect/Publish/Subscribe/Receive** sobre **parking/***). En producción: un certificado por dispositivo (Fleet Provisioning), renovación 12-24 m y AWS IoT Device Defender.

Transporte: TLS 1.2 obligatorio (puerto 8883), mTLS con CA AWS IoT, validación de **AmazonRootCA1.pem** en el dispositivo. API REST siempre por HTTPS.

Procesamiento: las tres Lambdas corren bajo **LabRole** (limitación del lab); en producción se sustituiría por un rol IAM por Lambda con políticas a medida (**ingest** solo **PutItem** sobre ***-state**, **aggregator Scan/PutItem**, **api** solo lectura). Sin credenciales en claro.

Persistencia: DynamoDB cifrado en reposo con KMS gestionado por AWS; en producción CMK propia. Point-in-Time Recovery con 35 días de retención.

Exposición: HTTPS + CORS controlado. Diseñado para producción: API Keys + Usage Plans (10 000 req/día por app), Cognito o JWT custom, AWS WAF y custom domain con ACM.

Operación: CloudWatch Logs + CloudTrail (audita toda la API AWS) + AWS Config (alerta de cambios en IAM o certificados).

7.3 Cumplimiento RGPD (privacy by design)

No se almacenan datos personales. Las cámaras ANPR procesan la matrícula **localmente en el gateway** y solo envían a la cloud un hash unidireccional y la dirección de paso; las imágenes originales se retienen un máximo de 72 h localmente. Las consultas a la API son anónimas. Retención: KPIs agregados 24 meses; logs operacionales a S3 Glacier tras 90 días.

7.4 Plan de respuesta a incidentes

1. **Detección:** alertas de CloudWatch o IoT Device Defender (intentos de conexión rechazados, picos de tráfico anómalos).
2. **Contención:** revocación inmediata del certificado afectado (`iot:UpdateCertificate` a `REVOKED`).
3. **Erradicación:** rotación de certificados afectados y sustitución física del sensor si hay sospecha de manipulación.
4. **Recuperación:** restauración del estado desde DynamoDB PITR si fue manipulado.
5. **Post-mortem:** revisión de logs CloudTrail y CloudWatch, informe a la concejalía y mejora de la policy o la lógica afectada.

8. Escalabilidad: del piloto a la ciudad

8.1 Piloto ($\approx$ 500 plazas)

Volumen: $\approx$ 150 000 mensajes/día (1,7 msg/s medio, pico 10 msg/s, $\sim$ 85 MB/día). Capacidad disponible: IoT Core soporta 30 000 pub/s ($\times$ 10 000 margen), DynamoDB on-demand absorbe 40 000 WCU, Lambda 1 000 concurrentes por defecto. Limitación observada: el **Scan** filtrado escala linealmente hasta $\sim$ 10 000 plazas; a partir de ahí migrar a **Query** con GSI.

8.2 Ciudad ($\approx$ 10 000 plazas)

Volumen: 50 000 cambios + 2 880 000 heartbeats = $\approx$ **3 M mensajes/día** (35 msg/s medio, pico 200 msg/s; 1,7 GB/día). Cambios arquitectónicos:

Componente	Cambio
Sensores	Onboarding con Fleet Provisioning Templates; certificado por dispositivo
Buffer	Interponer Amazon Kinesis Data Streams entre IoT Core y procesador
Procesamiento	Mantener Lambda o sustituir por un job de streaming si el volumen lo justifica
Estado	GSI <code>zoneId-status-index</code> en DynamoDB
Histórico	Política de TTL en <code>zone-kpis</code> o archivado periódico a S3
Datalake	S3 raw + Athena (analítica ad-hoc)
APIs	API Gateway con cuotas + CloudFront caché + WAF
HA	Multi-AZ por defecto; opcional DynamoDB Global Tables para DR multi-región

8.3 Particionado, alta disponibilidad y CI/CD

Particionado por `zoneId` (gestión municipal), distrito (reportes) y mes (S3 para Athena). Todos los servicios gestionados son multi-AZ por defecto; para DR multi-región se planificaría una segunda región pasiva con **DynamoDB Global Tables** y replicación de Lambda + API GW desplegable por CI/CD. Los sensores mantienen caché local de hasta 24 h y reenvío al recuperar conectividad. CI/CD recomendado: AWS CDK o Terraform, pipeline GitHub Actions / CodePipeline con despliegue blue/green y rollback automático, pruebas de carga trimestrales con Locust contra 5 000+ sensores simulados. Coste cloud agregado ciudad: $\approx$ **440 USD/mes**, equivalente a **4-6 céntimos por plaza/mes** (cap. 9).

8.4 Plan de pruebas a escala

Prueba	Objetivo	Métrica
Carga sostenida	35 msg/s constantes 24 h	p99 API < 2 s
Pico evento deportivo	1 000 eventos/min, 15 min	Sin throttling DDB/Lambda
Caída de IoT Core simulada	Buffering y reenvío correctos	0 pérdidas en cola < 24 h
Caída del agregador	Continuidad de la ingesta	KPIs eventualmente consistentes
Fuga de credencial sensor	Revocación rápida del certificado	< 5 min detección → revoc.

9. Análisis de costes

9.1 Hipótesis

Valores tomados como referencia de mercado para 2025-2026: EUR/USD = 1,08; vida útil sensor 6 años; cuadrilla 60 €/h con rendimiento 0,3 h/sensor (cifras habituales en pliegos municipales españoles); SIM NB-IoT M2M 0,60 €/SIM/mes a gran volumen (tarifas Movistar/Vodafone/Orange en contratos > 10 000 SIMs); gateway edge 700 € + 100 € instalación (precios MikroTik/Cisco IR1101); cámara ANPR con compute edge 900 € (Hikvision iDS/Axis Q-series). Tarifas AWS de aws.amazon.com/pricing (us-east-1, mayo 2026); en eu-west-1 $\approx +10\%$.

9.2 Piloto (500 plazas)

Concepto	Unid.	Ud (€)	Total (€)
Sensores AMR	500	75	37 500
Instalación sensores	500	40	20 000
Gateways edge (3 + redundancia)	4	800	3 200
Cámaras ANPR	8	900	7 200
Mástiles / soportes	8	250	2 000
Nodos ambientales	4	350	1 400
Sistema central respaldo	1	1 500	1 500
Subtotal hardware			72 800
Ingeniería + gestión (20 %)			14 560
CAPEX piloto			87 360 €

OPEX mensual piloto: SIM NB-IoT ($500 \times 0,60 \text{ €}$) = 300 €; AWS IoT Core (4,5 M msg) $\approx 4,2 \text{ €}$; Lambda $\approx 0,7 \text{ €}$; DynamoDB $\approx 7,5 \text{ €}$; API Gateway (1 M req) $\approx 3,3 \text{ €}$; S3/CloudWatch $\approx 2,4 \text{ €}$; mantenimiento ($10 \text{ h} \times 35 \text{ €/h}$) = 350 €. **Total $\approx 670 \text{ €/mes}$** (cloud puro $\approx 18 \text{ €/mes}$, muy por debajo de RNF-13).

9.3 Ciudad (10 000 plazas)

CAPEX ciudad: sensores 750 000 €, instalación 350 000 €, 80 gateways 64 000 €, 50 cámaras 45 000 €, 70 nodos ambientales 24 500 €, centro operaciones 8 000 € → subtotal 1 241 500 € + 15 % ingeniería = $\approx 1\,427\,725 \text{ €}$.

OPEX mensual ciudad: SIM 5 500 €, mantenimiento físico 6 000 €, AWS IoT Core 83 €, Kinesis 70 €, Lambda/DynamoDB/API/S3/CloudWatch $\approx 250 \text{ €}$, WAF + CloudFront $\approx 30 \text{ €}$, 1 FTE operaciones 3 500 € → $\approx 15\,440 \text{ €/mes}$ (cloud puro $\approx 440 \text{ €/mes} \approx 4\text{-}6 \text{ cént./plaza/mes}$).

9.4 TCO a 5 años

Escenario	CAPEX (€)	OPEX 5 a. (€)	TCO (€)	€/plaza/año
Piloto (500)	87 360	40 200	127 560	51
Ciudad (10 000)	1 427 725	926 400	2 354 125	47

Una plaza regulada española factura > 1 €/día (≈ 365 €/año en zona azul/verde según tarifas habituales de la OTA en ciudades medianas): con un incremento de rotación del 5 % o una mejora del 2 % en sanciones evitadas, la solución se autofinancia. Para contraste, el coste declarado del proyecto SmartSantander (2010-2015, $\sim 12\,000$ nodos heterogéneos) se situó en torno a 70-100 €/plaza/año amortizado, en línea con el rango aquí estimado.

9.5 Conclusiones del análisis de coste

1. La fracción cloud del coste es marginal frente al hardware y la operación física.
2. El coste cloud crece aproximadamente lineal con el número de plazas; el verdadero motor del TCO es la flota física.
3. NB-IoT como red elegida es decisiva en el OPEX: cualquier alternativa con cuota mensual > 2 €/SIM duplicaría el coste operativo.
4. La arquitectura serverless evita inversión en infraestructura cloud propia (sin EC2/RDS) y permite un piloto encuadrable en el presupuesto típico de un proyecto de innovación de un ayuntamiento mediano.

10. Prototipo funcional

10.1 Estructura y stack

```
prototipo/
  infra/ (01_setup_iot_core, 02_dynamodb, 03_lambda, 04_apigw, 99_teardown)
  simulator/ (parking_sensor.py, fleet_runner.py, certs/)
  lambdas/ (ingest, aggregator, api)
  api/openapi.yaml
  dashboard/streamlit_app.py
```

Stack: Python 3.13 + boto3 + paho-mqtt + Streamlit + Plotly. Despliegue real en AWS Academy Learner Lab (Account 583916379944, us-east-1).

10.2 Lanzamiento paso a paso

```
python -m pip install -r requirements.txt
Copy-Item .env.example .env

python infra\01_setup_iot_core.py # Things, certificado, policy
python infra\02_setup_dynamodb.py # tablas
python infra\03_setup_lambda.py # 3 Lambdas + IoT Rule
python infra\04_setup_api_gateway.py # API REST stage prod

python simulator\fleet_runner.py --num-spots 40 --duration 180 --heartbeat 20 --tick 2
python -m streamlit run dashboard\streamlit_app.py

python infra\99_teardown.py # limpieza obligatoria
```

10.3 Verificación end-to-end

Paso	Resultado real
Crear IoT Core	40 Things, certificado, policy, attach OK

Paso	Resultado real
Crear DynamoDB	2 tablas ACTIVE
Crear Lambdas + Rule	3 Lambdas, IoT Rule activa
Crear API GW	URL https://85fbp0svzc.execute-api.us-east-1.amazonaws.com/prod
Simulador 60 s	40 sensores conectados, eventos publicados
GET /spots	200 OK, 39 items (1 sensor en fallo intencional)
GET /zones	4 sub-zonas: Z1 40 %, Z2 60 %, Z3 40 %, Z4 22 %
GET /spots?format=geojson	FeatureCollection con 10 features Z1
GET /zones/Z1-CAMPUS/kpis?limit=5	5 entradas serie temporal

Latencia medida entre publicación en el simulador y refresco en el dashboard: **2-3 segundos**.

10.4 Visualización y consumo

El dashboard Streamlit muestra: cabecera con KPIs globales, mapa con cada plaza coloreada (verde libre, rojo ocupada, gris sin datos), tabla y gráfico de barras por sub-zona, serie temporal del % de ocupación por zona y tabla filtrable de detalle. La inspección AWS se hace en Consola → IoT Core → MQTT test client (suscrito a `parking/#`), DynamoDB → Explore y CloudWatch Logs.

Ejemplo de consumo por un sistema externo (vehículo, app, panel):

```
curl https://<api-id>.execute-api.us-east-1.amazonaws.com/prod/spots
curl '.../prod/spots?zone=Z2-DEPORTIVO'
curl '.../prod/spots?format=geojson'
curl .../prod/spots/ALB-Z1-001
curl .../prod/zones/Z1-CAMPUS/kpis?limit=20
```

Las capturas (consolas AWS + dashboard) se entregan en `memoria/diagramas/` siguiendo la nomenclatura `captura_*.png`.

11. Limitaciones, trabajo futuro y defensa

11.1 Limitaciones

Académicas: credenciales AWS Academy 3 h, solo `LabRole`, servicios fuera del catálogo del lab no siempre disponibles. **Técnicas:** certificado X.509 compartido (en producción uno por dispositivo), **Scan** filtrado en DynamoDB (futuro GSI), agregador en Lambda síncrona, sin alertas automáticas de sensor caído, sin autenticación en la API pública, dashboard local sin multi-usuario, una sola región. **Funcionales:** sin estimación de *dwell time*, sin predicción ML, sin gestión de plazas reservadas, sin panel ciudadano, sin paneles DMS.

11.2 Roadmap 12 meses

Mes	Hito
1	Piloto técnico con 20 sensores reales
2-3	Despliegue de los 500 sensores piloto
4	Pentest + endurecimiento (Cognito, WAF, Device Defender)
5	Endpoint NGSI-v2 opcional para interoperabilidad
6	Archivado del histórico y cuadros de mando ampliados
7	PWA ciudadana con guiado a plazas libres
8	Auditoría y alertas automáticas de sensores caídos
9	Integración con DMS municipales

Mes	Hito
10-12	Escalado a 10 000 plazas

11.3 Defensa: preguntas previsibles

- **¿Por qué AMR magnético?** Privacidad, consumo (5-7 años), coste (~75 €), robustez IP68; cámaras solo en accesos como conteo agregado.
- **¿Y si falla un sensor o gateway?** Sensor: heartbeat ausente detectado por auditoría `lastUpdated` (futura alerta SNS). Gateway: redundancia por zona. Conectividad: buffer local y reenvío.
- **¿Cómo evitar falsos libres/ocupados?** Debounce 2-3 s en edge, umbral `confidence`, calibración in situ, filtrado en la Lambda.
- **¿Cómo se escala a 10 000 plazas?** Kinesis como buffer, GSI en DynamoDB, WAF + CloudFront; coste cloud $\approx$ 440 €/mes.
- **¿Qué latencia se espera?** NB-IoT 1-3 s + IoT/Lambda/DDB 50-300 ms + API 100-500 ms; total < 5 s (medido 2-3 s).
- **¿Estado vs histórico?** `parking-state` sobrescribe; `zone-kpis` indexa por (`zoneId`, `windowEnd`); raw events a S3 (diseño).
- **¿Cómo proteges la API?** HTTPS, CORS, throttling y API keys, WAF y Cognito (producción), IAM mínimo privilegio.
- **¿Qué es real y qué simulado?** Real: AWS IoT Core, 3 Lambdas, 2 DynamoDB, API GW, dashboard. Simulado: sensores físicos (paho-mqtt sobre MQTT/TLS real), cámaras ANPR, gateway edge.
- **¿Por qué AWS y no FIWARE?** El enunciado exige AWS. FIWARE se contempla como posible capa de interoperabilidad opcional (cap. 5.7), no como sustituto.

11.4 Checklist de entrega

Memoria (.md, .tex, .pdf) en `memoria/`; prototipo ejecutable de cero con `README_GUIA.md`; capturas en `memoria/diagramas/`; `99_takedown.py` ejecutado; ZIP final sin credenciales (.aws/, `infra/infra_state.json` `simulator/certs/`); autor `alonso.marcos@alu.uclm.es`.